



Eindhoven University of Technology
Department of Mathematics and Computer Science
Software Engineering and Technology Research Group

Configurable Monitoring Infrastructure for ROS 2 Applications

Master's Thesis

Shikai Jin
s.jin1@student.tue.nl

Supervisor:
Dr. Ivan Kurtev

Eindhoven, June 2026

Abstract

Robot Operating System 2 (ROS 2) is widely used to build robotic applications. A ROS 2 application often contains many nodes that communicate through topics, services, and actions. This makes the application flexible, but it can also make its runtime behaviour difficult to understand. Existing monitoring solutions often focus on one communication type, one monitoring method, or one deployment setup. Developers therefore need to write extra monitoring code for each new application.

In this thesis, I present a configurable monitoring infrastructure for ROS 2 applications. I built it as a reusable foundation for collecting, processing, routing, checking, and exporting runtime data from heterogeneous ROS 2 interfaces. The infrastructure observes topics, service events, and action feedback and status, and converts these different observations into a common record format. Each record includes the observed data and useful context, such as its source, time, session, and identifier.

Users describe the monitoring setup in YAML configuration files. They can select data sources, reduce or filter data, choose outputs, and connect property-specific checking components. The prototype I implemented also supports plugins, so new data converters, verdict services, input sources, and output exporters can be added without changing the monitoring core.

The infrastructure runs in two ways. In the integrated setup, data collection and checking run together. In the split setup, data is collected near the robot and sent to a separate verifier. The concrete prototype supports file and MQTT transport and provides processing steps such as field selection, rate limiting, and change filtering.

I exercised the prototype through unit tests, Docker Compose deployments, offline replay, multi-robot examples, and a Navigation 2 case study. Across these scenarios, the same record format and plugin-based processing chain connected ROS 2 topics, service events, and action progress to reusable checking components in both integrated and split deployments.

Contents

Contents	2
1 Introduction	4
1.1 Context and Motivation	4
1.2 Problem Statement	4
1.3 Research Objective and Questions	5
1.4 Approach	5
1.5 Contributions	6
1.6 Thesis Outline	6
2 Background	7
2.1 ROS 2 Applications	7
2.1.1 Nodes and the ROS Graph	7
2.1.2 Topics	7
2.1.3 Services	8
2.1.4 Actions	8
2.1.5 Discovery, Middleware, and Quality of Service	8
2.2 Runtime Monitoring and Runtime Verification	8
2.2.1 Main Terms	8
2.2.2 Online and Offline Monitoring	9
2.2.3 Passive and Active Monitoring	9
2.2.4 Centralized and Distributed Monitoring	9
2.3 Main Monitoring Tasks	9
2.3.1 Collecting Data	9
2.3.2 Processing Data	9
2.3.3 Routing and Storage	10
2.3.4 Checking and Verdicts	10
2.3.5 Using the Results	10
2.4 Variability and Configuration	10
2.4.1 Feature-Oriented Domain Analysis	10
2.4.2 Configuration and Plugins	10
2.5 Deployment and Transport	10
2.5.1 Integrated Monitoring	10
2.5.2 Split Monitoring	11
2.5.3 MQTT Transport	11
3 Related Work	12
3.1 How the Approaches Are Compared	12
3.2 ROSMonitoring	12
3.3 ROMoSu	13
3.4 Generated Monitors for ROS 2	13

3.5	RTAMT	13
3.6	Tracing and Performance Monitoring	13
3.7	Network-Level Monitoring and Security	14
3.8	Containerized and Distributed Verification	14
3.9	Digital-Twin Runtime Verification	14
3.10	Field-Based Testing Guidance	14
3.11	Summary Comparison	15
3.12	Capability Matrix	16
3.13	Research Gap	17
4	Design	18
4.1	Feature Model as Design Input	18
4.2	Selected Feature Coverage	19
4.3	Design Requirements	20
4.4	Infrastructure Overview	20
4.5	Pipeline Roles	21
4.6	Common Data Record	22
4.7	Deployment Topologies	23
5	Implementation	25
6	Evaluation	26
7	Discussion	27
8	Conclusions	28
	Bibliography	29
	Appendix	30
A	Appendix	31

Chapter 1

Introduction

1.1 | Context and Motivation

Robots often work in changing environments. Their behaviour depends on software, hardware, sensor data, communication, and the physical world. Some events can only be seen while the robot is running. For this reason, testing a robot before deployment is important, but it is not always enough. Developers also need to observe the robot during operation.

Runtime monitoring collects information from a running system. The collected information can be used to find errors, understand runtime behaviour, or check whether the system follows expected rules [1, 2]. Unlike model checking, runtime monitoring does not study every possible system state. It studies the behaviour that actually happens during an execution.

 ROS 2 is a common platform for building robotic applications. A ROS 2 application is made of nodes that communicate through typed interfaces. Topics are mainly used for continuous data streams. Services are used for request-response communication. Actions are used for tasks that take longer and can provide feedback while they run [3, 4].

This modular structure makes it easier to reuse and replace software parts, but it also spreads the behaviour of the robot across many nodes and interfaces. Important information may come from several topics, services, or actions. It can therefore be difficult to understand what the full system is doing.

ROS 2 already provides several ways to observe a system. For example, a **monitor** can subscribe to a topic, read service introspection events, or observe the hidden topics used by actions. Lower-level tools can also trace ROS 2 execution. The difficult part is turning these possibilities into a reusable monitoring setup: developers still need to decide what to observe, how much data to keep, where to send the data, and how to check it. These decisions are often implemented again for every application.

During the pre-phase, I studied the monitoring options that ROS 2 makes available rather than starting from one fixed architecture. That work led me to compare observable elements, centralized and distributed architectures, active and passive monitoring, single- and multi-robot setups, and different implementation methods. The prototype in this thesis is the part of that wider space that I chose to make concrete: topics, service events, action feedback and status, configurable data processing, pluggable checking components, and two deployment setups.

1.2 | Problem Statement

A complete ROS 2 monitoring infrastructure can be viewed as a stack. At the bottom, it has to observe the running ROS 2 system and collect data. Above that, the collected observations need to be represented in a form that later components can understand. Only after that does property checking or other analysis become possible. This thesis is mainly about the foundation

of that stack and about the boundary where collected data is handed to checking components.

The difficulty is that ROS 2 does not expose all behaviour through one simple interface. Topics can be observed with an extra subscriber, but services need service introspection, and actions combine hidden topics with service-based goal and result communication. In the pre-phase report, this was the point where action monitoring stood out to me: a topic-only monitor would miss a large part of the behaviour of systems such as Navigation 2.

There is also no single right amount of data to collect. In one application, a complete message log may be useful for offline replay; in another, only a few fields from a high-rate stream matter. Saving too little data makes checking weak, while saving everything can make logs and transport payloads hard to use. This is why field selection, rate limiting, and change-based filtering are part of the design rather than later conveniences.

I also separate collection from checking because these two tasks change for different reasons. Collection depends on ROS 2 communication details and type information. Checking may be a threshold, a state machine, a temporal logic, or a project-specific service. The same separation is needed for deployment: sometimes the whole monitor can run near the robot, while in other cases a robot-side collector should send records to a separate verifier. Both cases need the same configuration ideas and the same record format.

Based on these problems, this thesis asks how a configurable ROS 2 monitoring foundation can collect different types of runtime data and send them to different checking components.

1.3 | Research Objective and Questions

The objective of this thesis is to design, implement, and **validate** a configurable monitoring infrastructure for ROS 2 applications. The infrastructure should collect different ROS 2 observations in a common way. It should also allow users to change the processing, checking, output, and deployment setup through configuration and **plugins**.

The following research questions guide the work:

- RQ1** What monitoring capabilities currently exist for ROS 2, and which observation gaps remain?
- RQ2** How can a feature model organize the observable elements and monitoring options of ROS 2?
- RQ3** How can a configurable infrastructure bring heterogeneous ROS 2 observations into one unified record and processing chain that keeps collection separate from specific checking methods?
- RQ4** To what extent can configuration and plugins be applied across the monitoring process, from source selection and data processing to checking, transport, and output?

RQ1 is answered through a review of existing research and tools. RQ2 uses the feature-oriented view developed during the pre-phase and the common concepts found in the implementation. RQ3 is answered by designing and implementing the common record and pipeline. RQ4 is answered by identifying the configurable and pluggable points in the prototype and exercising them through tests, deployment examples, offline replay, and a Nav2 case study.

1.4 | Approach

The thesis work followed the path that started in the **pre-phase report**. I first reviewed ROS and ROS 2 monitoring approaches and grouped them by the level at which they observe the

system: ROS graph instrumentation, network-level observation, and lower-level tracing. I then used those observations to refine the feature model and to decide which part of the configuration space was realistic to implement in the thesis prototype.

The prototype was developed in Python. Its runtime path is a pipeline rather than one monolithic monitor. A collector observes a configured topic, service event stream, or action feedback/status stream and stores each observation in a common data record. That record carries the payload together with source information, timestamps, a session identifier, and a record identifier. Optional transformers can then reduce or reshape the record before dispatchers send it to outputs or checking chains. When a property check is configured, a converter prepares the record for a verdict service, and verdict exporters send the result to a file, standard output, MQTT, or another configured destination.

The same pipeline can be used in two deployments. In the integrated deployment, all steps run together in one ROS 2-facing process. In the split deployment, the robot-side monitor collects and processes data. It sends the records through MQTT to a separate verifier, which runs the checking components without a ROS 2 dependency.

The prototype was developed together with unit tests, Docker Compose examples, and ROS 2 scenarios. A Navigation 2 (Nav2) case study is used because Nav2 uses topics, services, and actions in a realistic robotic application.

1.5 | Contributions

The contributions of this thesis are:

1. An overview of common ROS 2 monitoring capabilities, options, and gaps.
2. A monitoring architecture that separates collection, processing, transport, checking, and output.
3. A common record format for topics, service events, and action feedback and status.
4. A configuration and plugin model for changing monitoring behaviour without changing the monitoring core.
5. Support for both integrated and split monitoring deployments.
6. A validation of the prototype through tests, deployment examples, offline replay, multi-robot scenarios, and a Nav2 case study.

The main contribution is the reusable monitoring infrastructure. It is not a new checking algorithm. Different checking tools can be connected to the infrastructure through plugins.

1.6 | Thesis Outline

Chapter 2 explains the main ROS 2 and monitoring concepts. Chapter 3 reviews related monitoring and verification approaches. Chapter 4 presents how the feature model is used to select the prototype coverage and derive the infrastructure design. Chapter 5 describes the prototype. Chapter 6 presents the evaluation. Chapter 7 discusses the results and limitations. Chapter 8 concludes the thesis and describes future work.

Chapter 2

Background

This chapter explains the main concepts used in this thesis. It first introduces ROS 2 and its communication methods. It then explains runtime monitoring, the main tasks of a monitoring infrastructure, configuration, and deployment choices.

2.1 | ROS 2 Applications

2.1.1 Nodes and the ROS Graph

ROS 2 is a software platform for building robotic applications. It provides communication, discovery, build tools, and many reusable packages [3]. ROS 2 is not an **operating system** such as Linux. Instead, it runs above the operating system and helps software components work together.

A ROS 2 application is usually divided into *nodes*. Each node has a focused task. For example, one node may publish camera data, another node may plan a route, and another node may control the motors. The active nodes and their communication links form the *ROS graph*. Nodes can run on the same computer or on different computers.

ROS 2 communication uses defined message types. A type describes the fields in a message, **service, or action**. Client libraries such as `rclcpp` and `rclpy` provide the matching C++ and Python APIs. **Type information is also important for monitoring. A monitor needs the correct type before it can read and understand captured data.**

The ROS graph makes monitoring easier in some ways because a new monitoring node can often join the graph without changing the original application. The same modularity also spreads the behaviour of the full robot over many nodes, so a monitor must select useful interfaces and keep enough context to connect related events.

2.1.2 Topics

Topics provide publish-subscribe communication. A publisher sends typed messages to a topic name. Any number of subscribers can receive these messages. The publisher does not need to know the subscribers directly. Topics are often used for sensor data, robot state, movement commands, and diagnostics [4, 5].

A monitor can observe a topic by creating an extra subscription. This is a simple and passive method because the original communication path does not need to change. Common ROS 2 tools such as `ros2 topic echo` and `ros2 bag record` work in this way.

Topic monitoring still requires several choices. Some topics publish large messages or publish at a high rate, while a monitor may only need a few fields or a subset of the messages. It must also use a compatible Quality of Service setting, and messages from different publishers may arrive in a different order from the order in which they were sent.

2.1.3 Services

Services provide request-response communication. A client sends a request to a named service, and a server returns a response. Services are normally used for short operations that need a clear result [4].

An unrelated node cannot observe a normal service call by creating another client. ROS 2 therefore provides *service introspection*. When introspection is enabled, the client or server publishes event messages. These events can contain metadata, request data, and response data [6].

A monitor can subscribe to the event topic and create records for requests and responses. This allows passive observation, but introspection must first be enabled by the application. The monitor must also keep metadata that connects a response to the correct request.

2.1.4 Actions

Actions are used for tasks that take time to complete. An action client sends a goal to an action server. While the goal is running, the server can send feedback. The client can cancel the goal, and the server later returns a result. Navigation and robot manipulation are common examples [4, 7].

An action is built from hidden topics and services. Goal, result, and cancellation communication use services. Feedback and status use topics [7]. This makes action monitoring more difficult than normal topic monitoring.

A monitor can directly subscribe to action feedback and status. Observing goals, results, and cancellations also requires service introspection. The monitor should present these events under the logical action name instead of forcing later components to understand all hidden ROS 2 names.

2.1.5 Discovery, Middleware, and Quality of Service

ROS 2 commonly uses Data Distribution Service (DDS) middleware for communication. DDS provides distributed discovery, so ROS 2 does not need the central master used in ROS 1 [3]. Nodes can discover the names and types of visible topics and services.

Discovery can help a monitor find a message type automatically, but it is not a complete snapshot of the system. Discovery takes time, and the graph can change while the application is running, so a monitor should not assume that every source is already visible at startup.

ROS 2 also provides Quality of Service (QoS) settings. These settings control reliability, stored history, durability, deadlines, and other communication behaviour [8]. A monitoring subscriber needs compatible settings. Otherwise, it may receive no messages or lose data under high load.

2.2 | Runtime Monitoring and Runtime Verification

2.2.1 Main Terms

Runtime monitoring means observing a system while it runs. It can be used for many purposes, including operational diagnosis, error detection, security, and requirements checking [1].

Runtime verification is a related technique. It compares an observed execution with a specification [2]. It can show whether the observed behaviour follows or breaks a property; it does not normally prove that every possible future execution will be correct.

This thesis uses the following terms:

System under monitoring The ROS 2 application being observed.

Observation or event One captured runtime event, such as a topic message or service request.

Trace A sequence of observations used for later checking.

Property A rule or expected condition.

Monitor A component that observes the system or checks properties.

Verdict The result of a property check.

2.2.2 Online and Offline Monitoring

Online monitoring processes data while the system is running. It can report problems quickly and can support immediate feedback. The design question is where this monitoring logic should run and which observations need to be processed online.

Offline monitoring checks recorded data after the run has finished. It cannot react immediately, but the same data can be replayed many times and checked with checkers that are easier to run outside the robot application.

2.2.3 Passive and Active Monitoring

A passive monitor observes and reports. It does not change the behaviour of the application. An active monitor can respond to a problem, for example by blocking a message or starting a recovery action.

2.2.4 Centralized and Distributed Monitoring

In a centralized setup, observations are sent to one main checking component. Keeping the checking state in one place simplifies the design, although the central component can become a bottleneck or a single point of failure.

In a distributed setup, observation or checking work runs in several places. This can place monitoring logic closer to the data source or closer to a verifier, but it introduces questions about message ordering, clocks, network boundaries, and failures [9].

2.3 | Main Monitoring Tasks

A monitoring system usually performs several different tasks [1]. Keeping these tasks separate makes the infrastructure easier to configure and extend.

2.3.1 Collecting Data

The first task is to obtain data from the running system. In ROS 2, this can mean subscribing to topics, reading service events, observing action feedback, or using lower-level tracing tools.

Each method has different limits. Normal subscriptions are easy to add, but they only show what the monitor receives. Service introspection needs support from the client or server. Lower-level tracing can show detailed execution information but may not include the application meaning of a message.

2.3.2 Processing Data

Raw data is not always ready for checking. A monitor may need to select fields, rename values, add timestamps, or connect events to a session. It may also need to reduce the amount of data.

2.3.3 Routing and Storage

After processing, data must be sent to the correct destination. One **record** may be written to a file, sent through MQTT, and passed to a checking component. These outputs should not require changes to the **collector**.

Storage also supports later analysis. A stable record format makes it easier to replay a run or compare results from different checking tools.

2.3.4 Checking and Verdicts

Checking decides whether observed data follows an expected property. Different tools need different input formats. A simple threshold checker may need one number, while a temporal-logic checker may need a sequence of timestamped values.

2.3.5 Using the Results

A verdict is only useful when it reaches the right user or component. It may be printed during development, stored in a file, sent to a dashboard, or passed to another service.

2.4 | Variability and Configuration

2.4.1 Feature-Oriented Domain Analysis

Monitoring systems share many common tasks, but their exact needs differ. **Feature-Oriented Domain Analysis (FODA)** represents these common and variable choices as features [10]. A feature model can show required, optional, and alternative choices and can also describe rules between them.

2.4.2 Configuration and Plugins

Configuration allows users to change monitoring behaviour without editing the framework code. A monitoring configuration may describe what should be observed, how raw observations should be processed, where data should be sent, and which checking components should receive it.

Plugins handle choices that cannot be fully described by simple values. They allow a framework to delegate project-specific processing, checking, or output behaviour to separately developed components.

Configuration errors are still possible. A wrong topic type, field path, or plugin name can stop the monitor. Clear validation and error messages are therefore important parts of a configurable system.

2.5 | Deployment and Transport

2.5.1 Integrated Monitoring

In the integrated setup, collection, processing, checking, and output run together. This setup is simple and does not need a network connection between the monitor and verifier. It is useful for local development and smaller systems.

The price of this simplicity is tighter coupling: collection and checking share the same runtime context, so checker-specific failures or incompatible plugin behaviour can affect the full monitoring process.

2.5.2 Split Monitoring

In the split setup, a robot-side process collects and processes observations. A separate verifier receives the records and performs checking. This keeps ROS 2-specific collection close to the robot while placing checker-specific logic in a separate process.

This setup introduces a network boundary. Records may be delayed, lost, or received in a different order. The verifier also needs enough information in each record to understand its source and session.

2.5.3 MQTT Transport

MQTT is a publish-subscribe messaging protocol. Publishers send messages to named topics through a broker, and subscribers receive messages from the topics they select. MQTT is commonly used because it is simple and has wide tool support. MQTT 5.0 is an OASIS standard, and the protocol defines a broker-based model in which clients publish messages to topic names instead of connecting directly to each other [11].

This broker model fits the split monitoring deployment used in this thesis. The robot-side monitor publishes serialized data records to a configured MQTT topic or topic prefix. The verifier subscribes to that topic filter and receives the records without needing a ROS 2 dependency or a direct connection to the robot process. Verdicts can also be published to a separate MQTT topic, which keeps runtime data and checking results distinct.

MQTT also gives a configurable delivery trade-off through Quality of Service (QoS) levels. QoS 0 sends a message at most once and is useful when occasional loss is acceptable. QoS 1 retries until the broker acknowledges the message, which can duplicate records and therefore requires record identifiers to make duplicates recognizable. QoS 2 adds a stronger exactly-once exchange at the expense of a more involved protocol exchange. The infrastructure keeps the session and record identifier inside each data record so that MQTT transport does not have to carry the semantic identity of an observation.

Chapter 3

Related Work

There are many ways to monitor a robotic system. Some tools observe ROS messages, some generate checking code from requirements, and others trace the operating system or network. These approaches solve different parts of the monitoring problem. This chapter reviews the work that is most relevant to this thesis and explains how the infrastructure developed here differs.

3.1 | How the Approaches Are Compared

The word “monitoring” has a broad meaning. For example, a tracing tool may explain why a callback was slow, while a runtime-verification tool may report that a safety rule was broken. Both tools observe the running system, but their goals are different.

The comparison follows the questions that kept recurring during the pre-phase: which part of the system is observed, which ROS interfaces are covered, how properties are checked, what can be configured or extended, where collection and checking run, and what the approach is mainly trying to achieve.

These questions follow the general monitoring tasks identified by Rabiser et al., such as collection, processing, checking, routing, and using the results [1].

3.2 | ROSMonitoring

ROSMonitoring is a runtime-verification framework for ROS [12]. It adds monitor nodes between selected publishers and subscribers. A YAML file describes the topics that should be monitored. The monitor changes ROS messages into events and sends them to an external *oracle*. The oracle checks the events against a property.

The useful idea for this thesis is the boundary between ROS message handling and the external oracle. ROSMonitoring shows that the checker does not have to live inside the ROS-facing code. In this thesis, that boundary is combined with passive subscriptions and service introspection, so records can be routed through separate processing, transport, checking, and output stages without placing a proxy in the normal application communication path.

ROSMonitoring 2.0 adds support for services and studies message ordering [13]. Some service support is available for ROS 2. Full ROS 2 support and action monitoring are left as future work. Ordering is an important topic because messages may not arrive at the monitor in the same order in which they were published.

For this thesis, ROSMonitoring 2.0 is mainly a reminder that services and ordering become visible as soon as monitoring moves beyond plain topic streams. The prototype keeps ordering support modest and concentrates on a uniform record and pipeline for heterogeneous ROS 2 observations.

3.3 | ROMoSu

Stadler et al. describe several practical problems when adding monitoring to a ROS application. Developers need to understand the system, select useful data, choose collection rates, add checks, and present the results [14].

ROMoSu provides a flexible framework for these tasks [15]. Users can create reusable monitoring configurations, select topics and message fields, choose collection rates, and connect external services. ROMoSu also includes a user interface, a dashboard, and configuration storage.

ROMoSu is the closest work in spirit because it treats monitoring as something that should be configured, not hand-written from scratch each time. Its scope is broader on the user-facing side: it includes a dashboard, storage, and a larger platform around the monitoring process. The prototype here concentrates on the internal infrastructure stages - collectors, transformers, converters, verdict services, sources, and exporters - and gives direct attention to ROS 2 services, actions, and split robot/verifier deployment.

3.4 | Generated Monitors for ROS 2

Perez et al. present a process for generating ROS 2 runtime monitors from requirements [16]. The user writes structured requirements in FRET. These requirements are translated to temporal logic and then to Copilot monitor code through Ogma. The final monitor is compiled to C99 and placed in a generated ROS 2 package.

This is a requirements-to-monitor pipeline. The prototype starts one step earlier in the data path: it collects ROS 2 observations and prepares them for whatever checker is selected later. A generated FRET–Ogma–Copilot monitor could fit behind the verdict-service interface. The plugin examples implemented for this thesis are simpler custom converter and verdict-service classes.

3.5 | RTAMT

RTAMT is a library for monitoring Signal Temporal Logic (STL) properties [17]. It supports online and offline checking and can work with ROS 2. RTAMT can return a robustness value, which shows how strongly a signal satisfies or violates a property.

RTAMT helped clarify the role of converters in my design. STL monitoring needs timestamped signals, while another checker may want a different record shape. For that reason, the prototype keeps conversion separate from the verdict service. The thesis demonstrates that boundary with threshold-style and custom property plugins; an RTAMT-specific plugin remains future work.

3.6 | Tracing and Performance Monitoring

`ros2.tracing` provides low-overhead tracing for ROS 2 [18]. It uses LTTng and tracepoints in the ROS 2 core. Developers can study callback execution, message flow, latency, and operating system events.

`ros2.tracing` sits below the level used in this thesis. It is the right tool when the question is about callback execution, message flow, or middleware timing. The prototype stays with typed application data. In practice, the two levels are complementary: a property verdict can point to a suspicious run, and a trace can help explain the execution details behind it.

3.7 | Network-Level Monitoring and Security

ROS-FM observes ROS network traffic using eBPF and XDP [19]. It can collect network information, apply security rules, and support distributed visualization. Because it works below the ROS application layer, it can observe communication without adding a normal ROS subscriber for every topic.

This kind of monitor sees traffic without adding a ROS subscriber. The difficult part is reconstructing application meaning from packets. The design in this thesis stays at the application level, where ROS 2 messages already carry the type information that converters and checkers need.

ROSIDS23 and ROSPaCe provide datasets for ROS intrusion-detection research [20, 21]. ROSPaCe combines operating-system, network, and ROS 2 service data. These datasets show that security monitoring may need information from several system layers. Their main goal is security detection. In this chapter they mainly mark the boundary between security datasets and the more general monitoring infrastructure developed here.

3.8 | Containerized and Distributed Verification

Aldegheri et al. present a containerized verification environment for ROS [22]. Generated monitors run as ROS nodes in Docker containers. The work also studies where monitors should run between edge and cloud systems.

This work connects to the split deployment in this thesis because it treats placement as part of the monitoring design. The prototype here leaves placement as an explicit configuration choice: it provides integrated and split examples and keeps the transport interface replaceable.

Distributed runtime verification is a wider research field. It studies how monitoring can work when the monitored system and monitors are spread across different machines [9]. Important problems include clock differences, message delays, failures, and the division of a property between local monitors.

The split mode in this thesis is deliberately simpler than distributed runtime verification. Collection and checking can run on different machines, but checking is still logically centralized at the verifier.

3.9 | Digital-Twin Runtime Verification

Betzer et al. use a digital twin for runtime verification of an autonomous mobile robot under uncertainty [23]. Their monitors are synthesized from TeSSLa specifications and run with a cloud-located digital twin. The digital twin estimates the robot state, checks safety and performance properties, and can intervene when a property is about to be violated.

The interesting contrast is that the digital twin gives the checker a richer world model. The prototype in this thesis stays closer to the ROS 2 communication layer and builds the path from typed ROS 2 observations to checking components.

3.10 | Field-Based Testing Guidance

Caldas et al. provide guidance for runtime verification and field-based testing of ROS systems [24]. They explain that real-world conditions cannot always be fully reproduced in simulation. Robotic software should therefore support data collection, isolation, monitoring reactions, and the use of field evidence.

This paper helped frame the practical side of the record format. If field data is later used as evidence, the record needs enough context to connect an observation to a source, a session,

and a verdict. The prototype supports that kind of evidence trail through JSONL and MQTT outputs. The wider field-testing process, automatic recovery, and test generation remain outside its scope.

3.11 | Summary Comparison

Table 3.1 gives a short comparison of the main approaches.

Table 3.1: Comparison of selected related work.

Approach	Main focus	Checking and deployment	What I take from it
ROSMonitoring / 2.0	Intercepting ROS messages and services	External oracle; online and offline checking	Useful oracle boundary; this prototype keeps observation passive and includes action progress.
ROMoSu	Flexible and reusable ROS monitoring configurations	External services, user interface, and dashboard	Strong example of configurable monitoring; this work narrows in on the plugin-based pipeline underneath such a platform.
FRET-Ogma-Copilot	Generating monitors from structured requirements	Generated real-time checking code	Good candidate for a future verdict-service backend; not part of the current prototype.
RTAMT	Checking timestamped signals against STL properties	Online and offline STL monitoring	Motivates the converter boundary because STL checkers expect signal-shaped input.
<code>ros2_tracing</code>	Low-level ROS 2 and operating-system tracing	Trace analysis instead of property verdicts	Occupies a lower layer than the typed application records used here.
Digital-twin RV	Digital twin for robot monitoring under uncertainty	TeSSLa monitors with a digital-twin feedback loop	Shows how remote checking can use a richer state model; this thesis keeps the state source in ROS 2 records.
ROS-FM	Efficient network and security monitoring	Network metrics, security rules, and visualization	Useful contrast with application-level monitoring because packet data must be given ROS meaning again.

Table 3.1: Comparison of selected related work (continued).

Approach	Main focus	Checking and deployment	What I take from it
Containerized verification	Portable generated monitors and monitor placement	Docker and edge-to-cloud deployment	Informs the deployment view; the prototype leaves placement and transport as explicit configuration choices.
This thesis	Configurable collection of typed ROS 2 observations	Pluggable checking in integrated or split deployments	Connects topics, service events, and action progress to reusable processing, transport, checking, and output stages.

3.12 | Capability Matrix

The qualitative comparison above is complemented by two capability matrices. The symbols have the following meaning: ✓ means direct support, ● means limited support or support through a specific condition, and ✗ means that the capability is not a main feature of the approach. In the observable-element matrix, “Life.” refers to node lifecycle or node-state information, and “Kernel” refers to low-level operating-system or tracing events.

Table 3.2: Observable-element support in related work.

Approach	ROS version	Topics	Srv.	Act.	Par.	Life.	Kernel
ROSMonitoring	ROS 1	✓	✗	✗	✗	✗	✗
ROSMonitoring 2.0	ROS 1/2	✓	✓	✗	✗	✗	✗
ROMoSu	ROS 1	✓	✗	✗	✗	✗	✗
Generated ROS 2 monitors	ROS 2	✓	✗	✗	✗	✗	✗
ros2_tracing	ROS 2	✓	●	✗	✗	✓	✓
RTAMT	ROS 1/2	✓	✗	✗	✗	✗	✗
Digital-twin RV	ROS 1	✓	✗	✗	✗	✗	✗
This thesis	ROS 2	✓	✓	●	✗	✗	✗

Table 3.3: Checking formalism and execution mode in related work.

Approach	Formalism or checking mechanism	Online	Offline
ROSMonitoring	External oracle, for example Reelay	✓	✗

Table 3.3: Checking formalism and execution mode in related work (continued).

Approach	Formalism or checking mechanism	Online	Offline
ROSMonitoring 2.0	Runtime Monitoring Language and past-time MTL	✓	✓
ROMoSu	External services and Esper CEP	✓	✗
Generated ROS 2 monitors	FRETISH requirements and generated Copilot monitors	✓	✗
ros2.tracing	Trace collection and analysis, not a property language	✓	✓
RTAMT	Signal Temporal Logic and robustness monitoring	✓	✓
Digital-twin RV	TeSSLa specifications with a digital twin	✓	✗
This thesis	Configurable converter and verdict-service plugins	✓	✓

3.13 | Research Gap

After reviewing these approaches, the missing piece is not another property language. The gap is the infrastructure between ROS 2 communication and property-specific checking: a way to collect several kinds of ROS 2 observations, keep their context, process them uniformly, and then hand them to different checkers.

The combination pursued here is passive collection from topics, service events, and action feedback/status; one common record format with source, time, session, and identity information; configurable data reduction before storage or transport; and separate routing, checking, and output stages. Plugins then make the property-specific parts replaceable, while the same checking chain can be used in integrated and split deployments.

The contribution of this thesis is this combination as one reusable infrastructure. Its novelty lies in the uniform record format and pipeline that connect topics, service events, and action progress to the same configurable checking chain in both integrated and split deployments.

Chapter 4

Design

This chapter explains how I designed the monitoring infrastructure. The design starts from the feature model that I developed during the pre-phase. I used the model as a map of the design space rather than as an implementation checklist: it made clear which parts of ROS 2 monitoring could reasonably fit into one thesis prototype and which parts should remain outside the scope.

4.1 | Feature Model as Design Input

Figure 4.1 shows the feature model from the pre-phase. It organizes ROS 2 monitoring along several dimensions: observable elements, monitoring architecture, monitoring mode, system configuration, and implementation paradigm. When I built this model, the main lesson was that “monitoring ROS 2” is too broad to be treated as one feature. A monitor may observe topics, services, actions, parameters, node states, or middleware events. It may run centrally or in several places. It may be passive or active. It may be implemented as a normal ROS 2 monitor node, as an inserted decorator, as a network tap, or through lower-level middleware and tracing tools.

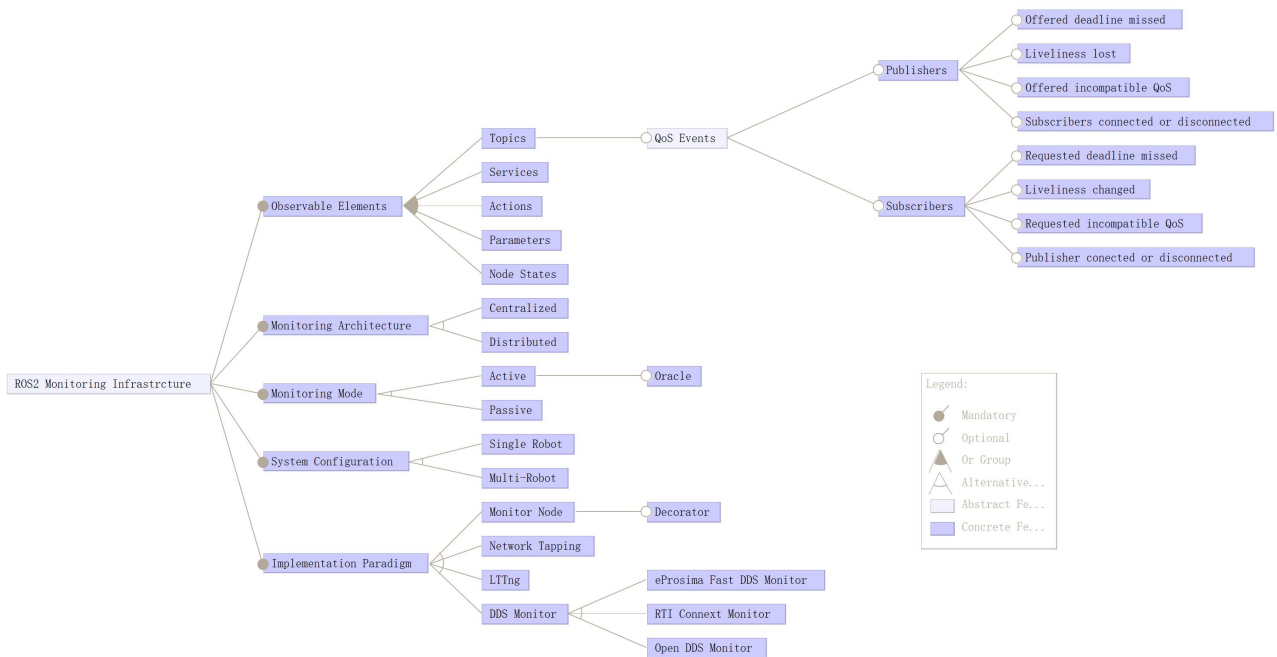


Figure 4.1: Feature model used during the pre-phase to explore the ROS 2 monitoring infrastructure domain.

The concrete prototype implements one path through this map: application-level data that can be collected passively, represented in a common format, and configured without rewriting the monitoring core.

4.2 | Selected Feature Coverage

Table 4.1 summarizes how the implemented prototype relates to the feature model. The table distinguishes between features that are implemented directly, features that are represented by the demonstration scenarios, and features that remain outside the current design.

Feature dimension	Selected coverage	Not covered in the prototype
Observable elements	Topics, service request and response events, and action feedback and status are collected as typed ROS 2 observations.	Parameters, node lifecycle states, DDS-level QoS events, network packets, and a complete action trace with goal, cancellation, and result events.
Monitoring architecture	The infrastructure supports an integrated setup and a split robot/verifier setup. Demo scenarios also exercise hybrid, multi-robot, global, choreographed, and offline replay organizations.	Automatic monitor placement, fault-tolerant distributed verification, global message ordering, and clock synchronization guarantees.
Monitoring mode	Monitoring is passive: the prototype observes data and produces records and verdicts.	Active intervention, automatic recovery, message blocking, or changes to robot behaviour.
System configuration	Single-robot and multi-robot configurations can be expressed with the same YAML configuration concepts and record format.	Automatic generation of the full monitoring configuration from a system model or graphical tool.
Implementation paradigm	The prototype is implemented as ROS 2-facing monitor nodes plus optional verifier-side processes.	Decorator-based interception, network tapping, LTTng tracing, and DDS-specific monitor implementations.

Table 4.1: Selected prototype coverage with respect to the feature model.

This coverage is intentionally narrower than the full feature model. The goal is not to cover every possible monitoring technique. The goal is to build and validate a reusable path from ROS 2 application observations to checking components, while keeping the parts that usually change between applications configurable.

4.3 | Design Requirements

The selected feature coverage led to a few design requirements that shaped the prototype. The most important one is the common record format. Topics, service events, and action feedback/status should not force later components to deal with three unrelated data shapes. The record therefore has to carry not only the payload, but also the source, timestamp, session, and identity information needed to connect observations with later verdicts.

I also wanted processing to happen early in the pipeline. Raw ROS 2 messages can be large or frequent, and in many cases the checker only needs a few fields or only needs records when values change. This pushed field extraction, throttling, and filtering into the infrastructure before storage or transport.

The remaining requirements are about keeping changeable parts apart. Collection needs ROS 2 type and communication knowledge without being tied to one checking language. A converter may adapt records for a checker, while a verdict service contains the property-specific logic. YAML configuration covers common choices, and plugins cover project-specific conversion, checking, input, and output behaviour. The same chain should then be usable both when everything runs in one ROS 2-facing process and when records cross a transport boundary to a separate verifier.

4.4 | Infrastructure Overview

The infrastructure is organized as a dataflow pipeline. A collector observes one ROS 2 source and creates a common data record. A transformer pipeline can reduce or modify the record, for example by selecting fields, limiting the rate, or emitting only changed values. Dispatchers then route the record to data exporters and to optional checking chains.

Checking is deliberately separated into two steps. A converter prepares a data record for one property-specific checker. A verdict service evaluates the prepared data and returns a structured verdict. Verdict exporters then deliver the result to standard output, a file, MQTT, or another configured destination. This separation keeps the ROS 2 collection layer independent from a particular verification language or checking algorithm.

Figure 4.2 shows the main pipeline stages. The same chain is used for topic messages, service events, and action feedback or status records, so a new source type does not require a new checking architecture.

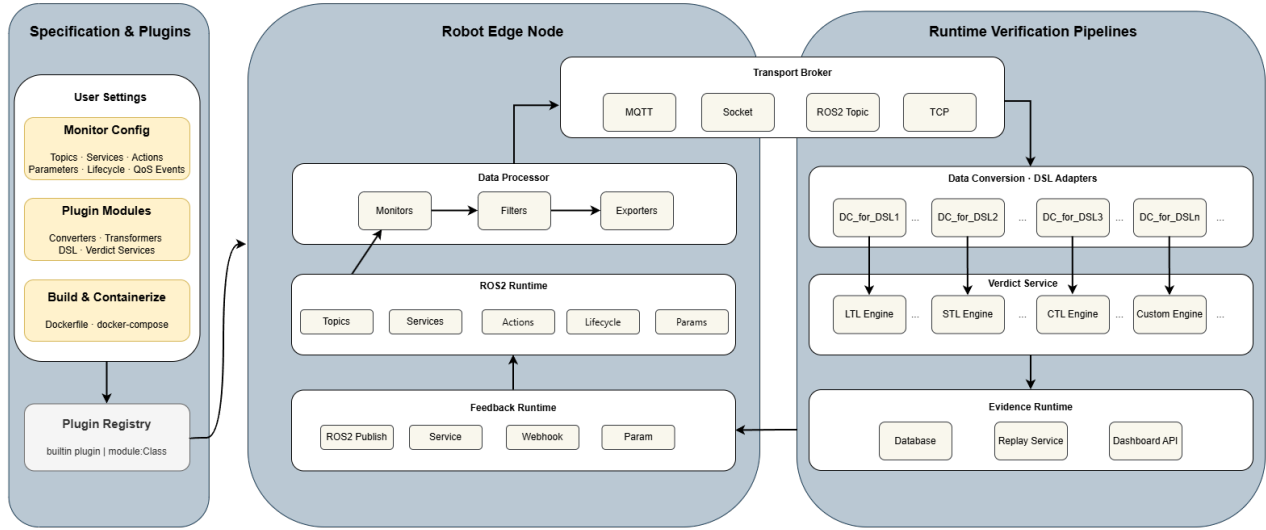


Figure 4.2: Main stages of the monitoring pipeline. Collectors create common data records, transformers process them, dispatchers route them, converters adapt them for checkers, and verdict exporters publish the checking results.

4.5 | Pipeline Roles

The implementation chapter uses several role names repeatedly. They have the following meanings in the design:

Collector A ROS 2-facing component that subscribes to one configured topic, service event stream, or action feedback/status stream and turns each observation into a data record.

Transformer pipeline An ordered set of processing steps that can remove fields, throttle records, or suppress unchanged values before data is stored, transported, or checked.

Dispatcher A routing component that fans a record out to one or more downstream exporters or checking chains.

Data exporter A sink for data records, such as a JSONL file, standard output, or MQTT topic.

Source A verifier-side input adapter that reads data records from a transport or replay file and injects them into the same checking chain.

Converter A property-specific adapter that projects a data record into the input shape expected by a checker.

Verdict service A checker that evaluates converted data and returns a structured verdict when a property state is observed.

Verdict exporter A sink for verdicts, such as standard output, a JSONL file, or an MQTT verdict topic.

This naming keeps the infrastructure precise: collectors and sources introduce records, transformers change records, dispatchers route records, converters adapt records for property logic, and verdict services perform the actual checking.

4.6 | Common Data Record

The common data record is the central abstraction of the infrastructure. Every captured topic message, service event, and action observation is converted into this format before it reaches processing, transport, or checking. Records are serialized as JSONL, with one JSON object per line. Empty optional fields are omitted from the serialized output to keep files and MQTT payloads compact.

Table 4.2 summarizes the fields used by the record. The field names follow the implementation and the detailed record specification in `docs/datarecord_spec.md`.

Field	Presence	Purpose
<code>_type</code>	All records	Identifies the record kind: session start, data, session end, or error.
<code>session_id</code>	All records	Groups all records produced during one monitoring run.
<code>record_id</code>	Data records	Gives one captured observation a stable identity for correlation with later verdicts.
<code>source_type</code>	Data records	States whether the observation came from a topic, service, or action.
<code>source_name</code>	Data records	Stores the logical ROS 2 name, such as <code>/cmd_vel</code> , <code>/set_bool</code> , or <code>/navigate_to_pose</code> .
<code>source_node</code>	When known	Records the publisher or server node name found during collection.
<code>msg_type</code>	Data records	Preserves the ROS 2 type identifier needed by downstream converters.
<code>phase</code>	Services/actions	Distinguishes request, response, feedback, status, goal, or result phases.
<code>timestamp</code>	All records	Stores the monitor-side wall-clock time when the record was created.
<code>ros_timestamp</code>	When available	Preserves the timestamp carried by the original ROS 2 message or event.
<code>data</code>	All records	Contains the monitoring configuration, captured payload, session statistics, or error details, depending on <code>_type</code> .
<code>metadata</code>	Data records	Stores infrastructure context such as sequence number, QoS profile, applied transformers, service event type, client identifier, and service sequence number.

Table 4.2: Schema of the common data record used throughout the infrastructure.

The record format is deliberately uniform across ROS 2 communication methods. Topics have no phase because each message is a complete unit. Service events are unwrapped into request and response records, with the service event metadata kept for correlation. Actions use feedback and status records directly, while goal and result phases can be represented when

service introspection for those action services is available. The same `session_id`, `record_id`, `source_type`, and `source_name` fields therefore identify observations regardless of whether they originated from a topic, service, or action.

4.7 | Deployment Topologies

The same pipeline can run in two main deployment forms. In the integrated deployment, collection, processing, checking, and output run in one ROS 2-facing process. This form is useful for local development and for properties that can be checked close to the observed application.

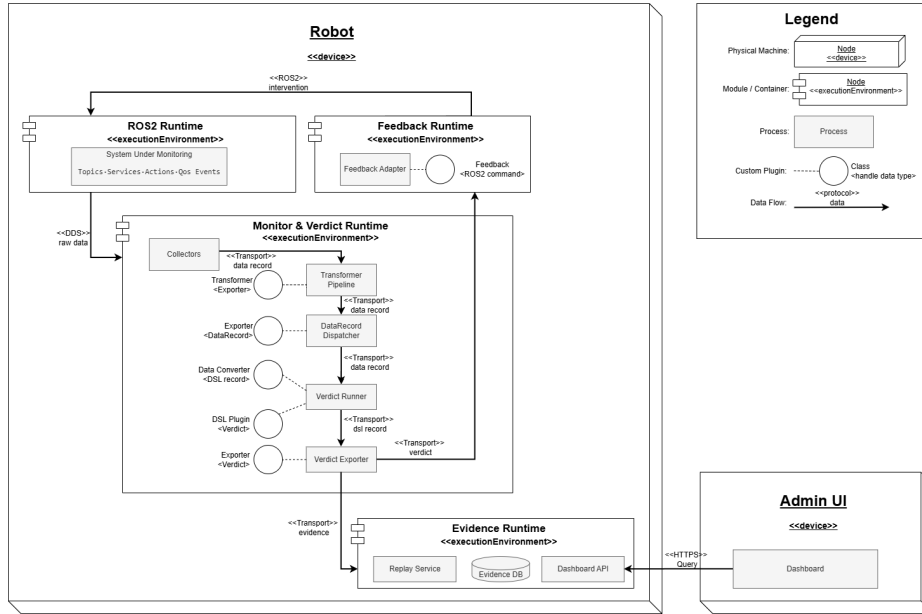


Figure 4.3: Integrated deployment topology. Collection, processing, checking, and verdict export are placed in the same ROS 2-facing monitor runtime.

In the split deployment, a robot-side monitor collects and processes observations and sends data records to a verifier-side process. The verifier uses a source adapter to read those records and then executes the same converter, verdict service, and verdict exporter chain that is used in the integrated deployment. The prototype implements file replay and MQTT for these flows, but transport remains an exporter/source choice rather than part of the checking logic.

Figure 4.5 shows the runtime order of the main pipeline interactions. The sequence is the same in both deployment forms until a data exporter or source crosses the transport boundary.

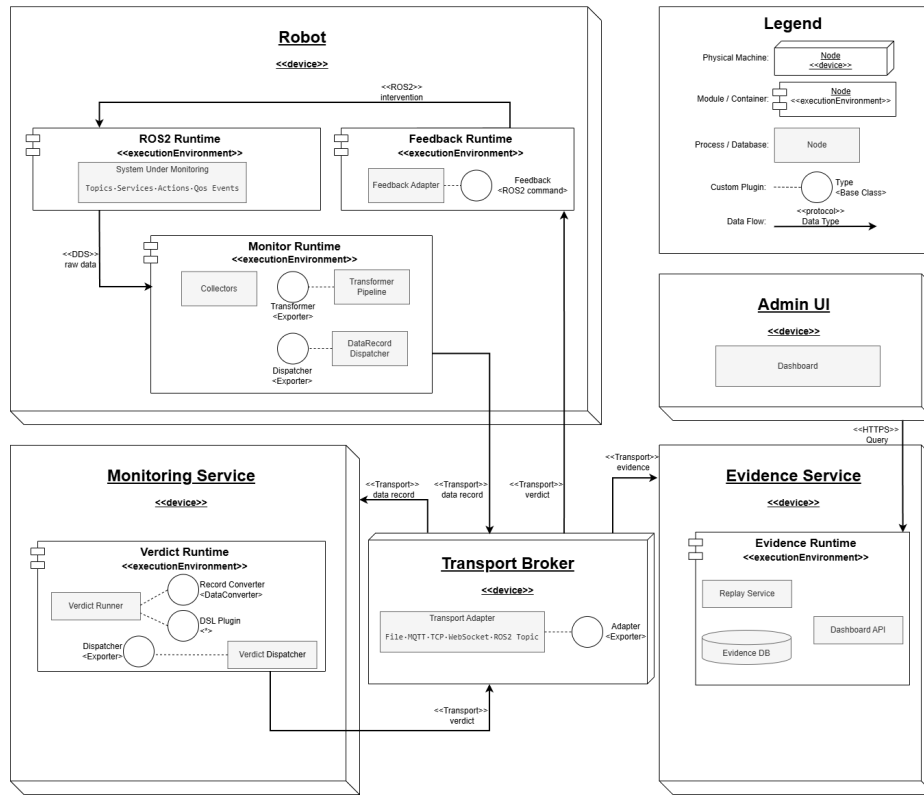


Figure 4.4: Split deployment topology. The robot-side monitor publishes data records through a transport boundary, while a verifier-side process runs the checking chain and exports verdicts.

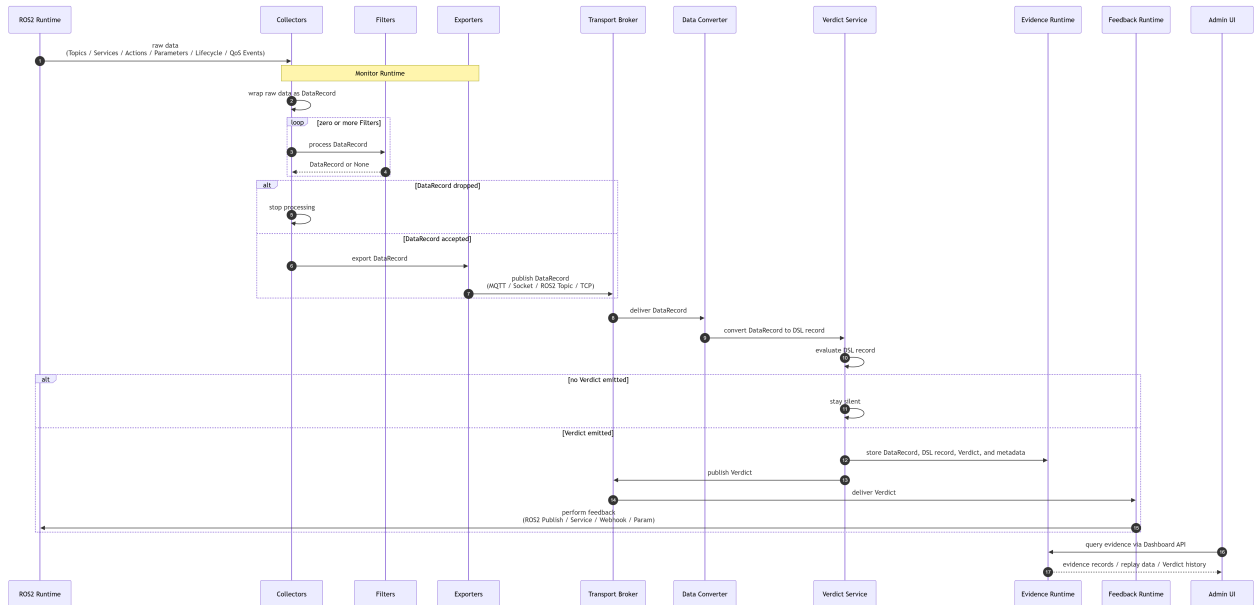


Figure 4.5: Runtime sequence from collection to verdict export.

Chapter 5

Implementation

Chapter 6

Evaluation

Chapter 7

Discussion

Chapter 8

Conclusions

Bibliography

- [1] Rick Rabiser, Klaus Schmid, Holger Eichelberger, Michael Vierhauser, Sam Guinea, and Paul Grünbacher. A domain analysis of resource and requirements monitoring: Towards a comprehensive model of the software monitoring domain. *Information and Software Technology*, 111:86–109, 2019. 4, 8, 9, 12
- [2] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. 4, 8
- [3] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. 4, 7, 8
- [4] Open Robotics. Topics vs services vs actions. ROS 2 Documentation. Accessed 2026-06-06. 4, 7, 8
- [5] Open Robotics. Ros 2 topics. ROS 2 Documentation. Accessed 2026-06-06. 7
- [6] Open Robotics. Configure service introspection. ROS 2 Documentation. Accessed 2026-06-06. 8
- [7] Geoffrey Biggs, Jacob Perron, and Shane Loretz. Actions. ROS 2 Design, 2019. Accessed 2026-06-06. 8
- [8] Open Robotics. Quality of service settings. ROS 2 Documentation. Accessed 2026-06-06. 8
- [9] Adrian Francalanza, Jorge A. Pérez, and César Sánchez. Runtime verification for decentralised and distributed systems. In Ezio Bartocci and Yliès Falcone, editors, *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 176–210. Springer, 2018. 9, 14
- [10] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-oriented domain analysis (FODA) feasibility study. Technical Report CMU/SEI-90-TR-021, Software Engineering Institute, Carnegie Mellon University, 1990. 10
- [11] OASIS MQTT Technical Committee. MQTT version 5.0. OASIS Standard, 2019. Accessed 2026-06-25. 11
- [12] Angelo Ferrando, Rafael C. Cardoso, Michael Fisher, Davide Ancona, Luca Franceschini, and Viviana Mascardi. ROSMonitoring: A runtime verification framework for ROS. In *Towards Autonomous Robotic Systems*, volume 12228 of *Lecture Notes in Computer Science*, pages 387–399. Springer, 2020. 12

- [13] Maryam Ghaffari Saadat, Angelo Ferrando, Louise A. Dennis, and Michael Fisher. ROS-Monitoring 2.0: Extending ROS runtime verification to services and ordered topics. In *Proceedings of the Sixth International Workshop on Formal Methods for Autonomous Systems*, volume 411 of *Electronic Proceedings in Theoretical Computer Science*, pages 38–55, 2024. 12
- [14] Marco Stadler, Michael Vierhauser, and Jane Cleland-Huang. Towards flexible runtime monitoring support for ROS-based applications. In *Proceedings of the 4th International Workshop on Robotics Software Engineering*, pages 12–15. ACM, 2022. 13
- [15] Marco Stadler and Michael Vierhauser. ROMoSu: Flexible runtime monitoring support for ROS-based applications. In *2023 IEEE/ACM 5th International Workshop on Robotics Software Engineering*, pages 53–60, 2023. 13
- [16] Ivan Perez, Anastasia Mavridou, Tom Pressburger, Alexander Will, and Patrick J. Martin. Monitoring ROS2: From requirements to autonomous robots. In *Proceedings of the Fourth Workshop on Formal Methods for Autonomous Systems*, volume 371 of *Electronic Proceedings in Theoretical Computer Science*, pages 208–216, 2022. 13
- [17] Tomoya Yamaguchi, Bardh Hoxha, and Dejan Ničković. RTAMT: Runtime robustness monitors with application to CPS and robotics. *International Journal on Software Tools for Technology Transfer*, 26(1):79–99, 2024. 13
- [18] Christophe Bédard, Ingo Lütkebohle, and Michel R. Dagenais. `ros2.tracing`: Multipurpose low-overhead framework for real-time tracing of ROS 2. *IEEE Robotics and Automation Letters*, 7(3):6511–6518, 2022. 13
- [19] Sean Rivera, Antonio Ken Iannillo, Sofiane Lagraa, Clément Joly, and Radu State. ROS-FM: Fast monitoring for the robotic operating system (ROS). In *2020 25th International Conference on Engineering of Complex Computer Systems*, pages 15–24, 2020. 14
- [20] Elif Değirmenci, Yunus Sabri Kırca, İlker Özçelik, and Ahmet Yazıcı. ROSIDS23: Network intrusion detection dataset for robot operating system. *Data in Brief*, 51:109739, 2023. 14
- [21] Tommaso Puccetti, Simone Nardi, Cosimo Cinquilli, Tommaso Zoppi, and Andrea Ceccarelli. ROSPaCe: Intrusion detection dataset for a ROS2-based cyber-physical system and IoT networks. *Scientific Data*, 11(1):481, 2024. 14
- [22] Stefano Aldegheri, Nicola Bombieri, Samuele Germiniani, Federico Moschin, and Graziano Pravadelli. A containerized ROS-compliant verification environment for robotic systems. In *2021 Design, Automation and Test in Europe Conference and Exhibition*, pages 222–225, 2021. 14
- [23] Joakim Schack Betzer, Jalil Boudjadar, Mirgita Frasheri, and Prasad Talasila. Digital twin enabled runtime verification for autonomous mobile robots under uncertainty, 2024. 14
- [24] Ricardo Caldas, Juan Antonio Piñera García, Matei Schiopu, Patrizio Pelliccione, Genáina Rodrigues, and Thorsten Berger. Runtime verification and field-based testing for ROS-based robotic systems. *IEEE Transactions on Software Engineering*, 50(10):2544–2567, 2024. 14

Appendix A

Appendix